

Fakultät Informatik

Entwicklung und Konzeption eines Device-Integration-Plugins für zenLAB

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Lukas Spindler

Matrikelnummer 3728783

Erstgutachter:	Prof. Dr. Matthias Teßmann
Zweitgutachter:	Prof. Dr. Matthias Meitner
Betreuer:	Jonas Krammer
Unternehmen:	infoteam Software AG

© 2026

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Kurzdarstellung

Kurze Zusammenfassung der Arbeit, höchstens halbe Seite. Deutsche Fassung auch nötig, wenn die Arbeit auf Englisch angefertigt wird.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Inhaltsverzeichnis

Vorwort	1
1. Einleitung	2
1.1. Problemstellung	3
1.2. Forschungsfragen	4
1.3. Zielsetzung und Abgrenzung	5
1.4. Aufbau der Arbeit	6
2. Grundlagen	7
2.1. zenLAB	7
2.1.1. Middleware- und Plugin-Architektur	7
2.1.2. DeviceAgents	8
2.2. Grundlagen verteilter Kommunikation	9
2.2.1. Auftrags- und Ergebnisprinzip	9
2.2.2. Korrelation, Timeouts und Abbruch	10
2.2.3. Adressierung und Routing	10
2.2.4. Asynchrone und bidirektionale Streams	10
2.3. Dynamische Dienste und Ereignisse	11
2.3.1. Registrierung	11
2.3.2. Ereignisse und Abonnements	11
2.4. Architektur- und Technologiegrundlagen	12
2.4.1. Stabile Verträge und Modularisierung	12
2.4.2. gRPC und SignalR	12
2.4.3. Dependency Injection und Testbarkeit	13
2.5. Zusammenfassung	13
3. Anforderungsanalyse	14
3.1. Ausgangssituation	14
3.1.1. Analyse der bestehenden Kommunikationsarchitektur	14
3.1.2. Grenzen der bisherigen Serviceanbindung	14
3.2. Zielsystem und Rahmenbedingungen	15
3.2.1. Beteiligte Akteure und Komponenten	15
3.2.2. Bestehende Client- und WebAPI-Grenze	15
3.2.3. Minimaler Funktionsumfang	15

3.3. Funktionale Anforderungen	15
3.3.1. Verfügbarkeit und eindeutige Identifikation	15
3.3.2. Verwaltung von Integrationen und Devices	16
3.3.3. Methodenstreams, Verbindungsabbruch und erneute Anbindung	16
3.3.4. Routing von Client-Aufträgen	16
3.3.5. SmartProxy und gRPC-Servicevertrag	17
3.3.6. Bereitstellung von Gerätefunktionen	17
3.3.7. Abonnieren von Wertänderungen	17
3.4. Nicht-funktionale Anforderungen	17
3.4.1. Wiederverwendbarkeit	17
3.4.2. Erweiterbarkeit	17
3.4.3. Zuverlässigkeit	17
3.4.4. Nebenläufigkeit	18
3.4.5. Performance	18
3.5. Abgrenzung	18
3.6. Zusammenfassung der Anforderungen	18
4. Konzeption und Implementierung	19
4.1. Ziel und Gesamtarchitektur	19
4.2. Technologische Rahmenbedingungen und Projektstruktur	19
4.2.1. Technologische Rahmenbedingungen	19
4.2.2. Projekt- und Paketstruktur	19
4.3. Auswahl und Nutzung des Gerätemodells	20
4.4. Verfügbarkeits- und Lebenszyklusverwaltung	20
4.4.1. Service Directory und Gerätecatalog	20
4.4.2. Methodenstream-Tracking	21
4.4.3. Streamabbruch, erneute Anbindung und Bereinigung	21
4.5. Auftragsvermittlung und Multi-Service-Routing	22
4.5.1. Routing-Grundlage	22
4.5.2. Serviceidentifikation und Routenauflösung	22
4.5.3. Job-Korrelation	22
4.5.4. Fehler- und Timeoutbehandlung	23
4.5.5. Implementierung der Routing-Grundlage und des Multi-Service-Routings	23
4.6. Contracts und Dienstschnittstellen	23
4.6.1. Job-, Ergebnis- und Stream-Contracts	23
4.6.2. Serviceverträge und SmartProxys	23
4.6.3. Code-first gRPC und Umsetzung der Dienstschnittstellen	23
4.7. Integration Client und Anbindung bestehender DeviceAgents	24
4.7.1. Adapter für das gewählte Gerätemodell	24
4.7.2. Implementierung des Integration Clients und des Adapters	24

4.8. Subscriptions und Ereignisweitergabe	24
4.8.1. Prototypische Umsetzung der Subscriptions	25
4.9. Einbindung in zenLAB und Referenzintegration	25
4.9.1. Einbindung in zenLAB	25
4.9.2. Anbindung eines Referenz-DeviceAgents	25
4.10. Herausforderungen während der Umsetzung	25
4.11. Begründung zentraler Entwurfsentscheidungen	25
5. Evaluation	26
5.1. Ziel der Evaluation	26
5.2. Versuchsaufbau und Evaluationskriterien	26
5.3. Überprüfung der funktionalen Anforderungen	26
5.3.1. Verfügbarkeit und eindeutige Adressierung	26
5.3.2. Routing mit mehreren Integrationen und Clients	26
5.3.3. Aktivierung und Abbruch erforderlicher Methodenstreams	26
5.3.4. Gerätezugriffe und Subscriptions	26
5.3.5. Kompatibilität der bestehenden Client- und WebAPI-Grenze	26
5.4. Bewertung der Wiederverwendbarkeit und Erweiterbarkeit	27
5.5. Messung des Routing-Overheads	27
5.6. Bewertung anhand des Referenz-DeviceAgents	27
5.7. Diskussion der Ergebnisse	27
5.8. Grenzen der Evaluation	27
6. Fazit und Ausblick	28
6.1. Zusammenfassung der Ergebnisse	28
6.2. Beantwortung der Forschungsfragen	28
6.3. Bewertung der Zielerreichung	28
6.4. Limitationen der Arbeit	28
6.5. Ausblick auf mögliche Erweiterungen	28
A. Ergänzende zenLAB-Architektur	29
Abbildungsverzeichnis	31
Tabellenverzeichnis	32
List of Listings	33
Literaturverzeichnis	34
Glossar	36

Vorwort

Im Rahmen meines Verbundstudiums bei der infoteam Software AG konnte ich bereits vor Beginn meiner Bachelorarbeit in der Abteilung „Life Science“ wertvolle Einblicke in die Entwicklung von Komponenten für zenLAB gewinnen. Dabei mein besonderes Interesse an der Weiterentwicklung dieser Softwarelösung entstanden. Da die Anbindung von Geräten in der Medizintechnik eine zentrale und zugleich anspruchsvolle Aufgabe darstellt, beschäftigt sich diese Bachelorarbeit mit der Erweiterung der bestehenden Software um ein Device-Integration-Plugin. Ziel der Arbeit ist es, eine technische Grundlage zu schaffen, durch die neue Geräte künftig einfacher, flexibler und strukturierter in zenLAB integriert werden können.

An dieser Stelle möchte ich mich herzlich bei Jonas Krammer und Dr. Melanie Kahl für die Betreuung im Unternehmen, die fachliche Unterstützung sowie das hilfreiche Feedback während der Erstellung meiner Bachelorarbeit bedanken. Mein Dank gilt außerdem dem gesamten zenLAB Team für die interessanten Gespräche und den konstruktiven Austausch zur Entwicklung und Integration des Device-Integration-Plugins. Insbesondere durch gemeinsame Diskussionen konnten unterschiedliche Perspektiven eingebracht, Herausforderungen frühzeitig erkannt und mögliche Lösungsansätze weiterentwickelt werden.

Darüber hinaus danke ich Fabio Starke für seine Überarbeitungsvorschläge zur schriftlichen Ausarbeitung. Ebenso möchte ich mich bei Prof. Dr. Matthias Teßmann von der Technischen Hochschule Nürnberg für die Betreuung meiner Bachelorarbeit, die Beantwortung meiner Fragen sowie das wertvolle Feedback bedanken.

In der vorliegenden Arbeit werden Eigenbegriffe, Markennamen von Unternehmen oder staatlichen Einrichtungen sowie Begriffe aus dem Glossar bei ihrer ersten Nennung kursiv hervorgehoben. Codeausschnitte, beispielsweise Klassen- oder Methodennamen, werden im Fließtext farblich gekennzeichnet. Aus Gründen der besseren Lesbarkeit wird in dieser Arbeit das generische Maskulinum verwendet. Dieses ist geschlechtsneutral zu verstehen und bezieht sich gleichermaßen auf Personen jeglichen Geschlechts.

Kapitel 1.

Einleitung

Laborprozesse werden zunehmend digitalisiert, vernetzt und automatisiert. Geräte, Softwaredienste und Benutzeranwendungen sollen dabei nicht mehr isoliert arbeiten, sondern gemeinsam in durchgängige Abläufe eingebunden werden. Diese Entwicklung wird häufig unter dem Begriff Labor 4.0 zusammengefasst. Sie verfolgt das Ziel, Laborumgebungen flexibler, transparenter und effizienter zu gestalten [1]. Eine wesentliche Voraussetzung dafür ist, dass unterschiedliche Laborgeräte zuverlässig in übergeordnete Softwaresysteme integriert und von mehreren Anwendungen genutzt werden können.

Die praktische Umsetzung wird dadurch erschwert, dass Laborgeräte unterschiedliche Kommunikationsschnittstellen, Protokolle und Funktionsmodelle besitzen. Übergeordnete Anwendungen sollten diese technischen Besonderheiten jedoch nicht für jedes Gerät erneut berücksichtigen müssen. Benötigt wird daher eine vermittelnde Softwareebene, welche gerätespezifische Unterschiede kapselt und die verfügbaren Gerätefunktionen in einer einheitlichen Form zugänglich macht. Auch bestehende Ansätze zur Laborautomation zeigen, dass die Erkennung und zentrale Verwaltung verfügbarer Geräte und Dienste eine wichtige Grundlage für automatisierte Arbeitsabläufe darstellt [2].

Die vorliegende Arbeit entsteht in Zusammenarbeit mit der infoteam Software AG. Das Softwareunternehmen entwickelt kundenspezifische Lösungen unter anderem für die Bereiche Industry, Infrastructure, Life Science und Public Service. Ein Schwerpunkt liegt auf Software für regulierte Medizin- und Laborprodukte [3]. Im Bereich Life Science bietet infoteam mit zenLAB ein Middleware-Framework für vernetzte digitale Labore an [4].

zenLAB bildet den Anwendungskontext dieser Arbeit. Innerhalb der Middleware übernehmen sogenannte DeviceAgents die Kommunikation mit den angebundenen Geräten und kapseln deren hersteller- und protokollspezifische Besonderheiten. Auf diese Weise können unterschiedliche Geräte über gemeinsame Schnittstellen angesprochen werden. Mit einer wachsenden Zahl unabhängig betriebener DeviceAgents entsteht jedoch die zusätzliche Aufgabe, verfügbare Integrationen und Geräte zentral zu erfassen, ihre Erreichbarkeit zu überwachen und Zugriffe verschiedener Clients zuverlässig an das jeweils zuständige Ziel zu vermitteln.

An diesem Punkt setzt die vorliegende Arbeit an. Ihr Gegenstand ist nicht nur die Weiterleitung einzelner Aufträge, sondern die Konzeption und prototypische Implementierung eines vollständigen Device-Integration-Plugins für zenLAB. Das Plugin soll die zentrale Verbindung zwischen Clientanwendungen und den bestehenden DeviceAgents bilden. Dazu gehören die Registrierung von Integrationen, die Verwaltung ihrer Geräte, die Überwachung ihres Lebenszyklus, die Vermittlung von Gerätezugriffen und die Weitergabe abonniertter Änderungen. Die gerätespezifische Kommunikation verbleibt weiterhin in den DeviceAgents.

1.1. Problemstellung

Die bestehende zenLAB-Architektur trennt die gerätespezifische Kommunikation bereits von den aufrufenden Anwendungen. Ein DeviceAgent kennt das angeschlossene Gerät und stellt dessen Funktionen bereit, während Clients über eine gemeinsame Schnittstelle auf diese Funktionen zugreifen. Diese Trennung erleichtert die Anbindung heterogener Geräte. Sie beantwortet jedoch noch nicht vollständig, wie mehrere zur Laufzeit hinzukommende Integrationen gemeinsam verwaltet und gleichzeitig von mehreren Clients genutzt werden können.

Ohne eine zentrale Integrationskomponente fehlt eine aktuelle Gesamtsicht auf die verfügbaren Integrationen und deren Geräte. Beim Hinzufügen, Neustarten oder Ausfall einer Integration muss nachvollziehbar bleiben, welche Geräte erreichbar sind. Eine erneute Verbindung darf nicht mit veralteten Zuständen der vorherigen Verbindung vermischt werden. Ebenso müssen gleichzeitige Clientzugriffe eindeutig dem richtigen Gerät zugeordnet und Ergebnisse an den jeweils auslösenden Aufruf zurückgegeben werden. Für abonnierte Geräteänderungen ist darüber hinaus eine dauerhafte und kontrollierbare Weitergabe an interessierte Clients erforderlich.

Die Herausforderung umfasst damit mehrere zusammenhängende Verantwortlichkeiten. Erstens müssen Integrationen eindeutig registriert und ihre Gerätelisten aktuell gehalten werden. Zweitens muss der Zustand einer Integration überwacht werden, damit Ausfälle erkannt und zugehörige Ressourcen vollständig bereinigt werden können. Drittens müssen Gerätezugriffe, Antworten und Ereignisse auch bei parallelen Abläufen korrekt zugeordnet werden. Schließlich soll sich die Lösung in die bestehende zenLAB-Architektur einfügen, ohne die etablierten Clientzugriffe oder die gerätespezifischen Implementierungen der DeviceAgents grundlegend zu verändern.

Hierfür soll ein Device-Integration-Plugin als zentrale Vermittlungs- und Verwaltungskomponente entwickelt werden. Das Plugin führt Informationen über Integrationen und

Geräte zusammen, stellt ihre Funktionen für Clients bereit und behandelt Änderungen während der Laufzeit. Die hierfür benötigten Mechanismen sind nicht als isolierte Routinglösung zu betrachten, sondern als Bestandteile einer übergeordneten Geräteintegration. Registrierung, Lebenszyklusverwaltung, Gerätezuordnung, Auftragsvermittlung und Subscriptions müssen deshalb gemeinsam konzipiert und aufeinander abgestimmt werden.

1.2. Forschungsfragen

Aus der Problemstellung ergibt sich die übergeordnete Frage, wie ein Device-Integration-Plugin gestaltet werden kann, das mehrere heterogene und dynamisch verfügbare Geräteintegrationen zuverlässig in zenLAB einbindet. Die Arbeit folgt dabei den drei aufeinander aufbauenden Schritten Anforderungsanalyse, Entwicklung und Evaluation.

Im Rahmen der Arbeit werden folgende Forschungsfragen untersucht:

1. Welche funktionalen und nicht-funktionalen Anforderungen ergeben sich für ein Device-Integration-Plugin zur zuverlässigen Einbindung dynamischer Integrationen und ihrer Geräte in zenLAB?
2. Wie lässt sich ein solches Plugin auf Grundlage dieser Anforderungen konzipieren, prototypisch implementieren und in die bestehende Client- und DeviceAgent-Architektur einfügen?
3. Inwieweit erfüllt der entwickelte Prototyp die ermittelten Anforderungen, insbesondere hinsichtlich Funktionalität, Zuverlässigkeit, Integrationsaufwand und Performance?

Die erste Forschungsfrage ermittelt, welche Aufgaben und Qualitätsmerkmale das Plugin erfüllen muss. Betrachtet werden insbesondere die Einbindung mehrerer Integrationen, die Verwaltung und Aktualisierung ihrer Geräte, die Überwachung ihrer Verfügbarkeit, der Umgang mit Verbindungsabbrüchen und erneuter Anbindung sowie die Nutzung durch mehrere Clients. Hinzu kommen Anforderungen an Fehlerbehandlung, Nebenläufigkeit, Erweiterbarkeit und die Einbindung in die vorhandene Systemlandschaft.

Die zweite Forschungsfrage überführt diese Anforderungen in eine Architektur und einen lauffähigen Prototyp. Im Mittelpunkt stehen die Verantwortlichkeiten und Schnittstellen des Device-Integration-Plugins sowie das Zusammenspiel von Geräteverwaltung, Lebenszyklusüberwachung, Auftragsvermittlung und Subscriptions. Die technische Ausgestaltung wird aus den Anforderungen abgeleitet und in die bestehende zenLAB-Architektur integriert.

Die dritte Forschungsfrage bewertet die entwickelte Lösung anhand der zuvor festgelegten Anforderungen. Untersucht werden sowohl reguläre Abläufe mit mehreren Integrationen und Clients als auch Fehlerfälle wie doppelte Kennungen, nicht erreichbare Integrationen, abgebrochene Methodenstreams und deren erneuter Aufbau. Zusätzlich werden der notwendige Anpassungsaufwand für einen bestehenden DeviceAgent und der durch die zusätzliche Vermittlung entstehende Kommunikationsaufwand betrachtet.

1.3. Zielsetzung und Abgrenzung

Ziel der Arbeit ist ein prototypisch funktionsfähiges Device-Integration-Plugin, das mehrere Integrationen und die von ihnen bereitgestellten Geräte zentral in zenLAB verfügbar macht. Der Eigenbeitrag umfasst die Ermittlung der Anforderungen, den Entwurf der Plugin-Architektur, die Implementierung der zentralen Komponenten und die anschließende Evaluation. Das Routing von Aufträgen ist dabei ein notwendiger Teil der Lösung, bildet jedoch nicht allein den Gegenstand der Arbeit.

Der Prototyp soll folgende Kernfunktionen bereitstellen:

- mehrere Integrationen eindeutig registrieren und verwalten,
- die jeweils bereitgestellten Geräte erfassen und bei Änderungen aktualisieren,
- die Verfügbarkeit angebundener Integrationen überwachen,
- Streamabbruch, Ausfall und erneute Anbindung konsistent behandeln,
- Gerätezugriffe gezielt an die zuständige Integration vermitteln,
- parallele Aufträge und Ergebnisse zuverlässig zuordnen,
- ausgewählte Geräteänderungen abonnieren und an Clients weitergeben sowie
- mindestens einen bestehenden DeviceAgent mit begrenztem Anpassungsaufwand anbinden.

Die Umsetzung wird mit mehreren gleichzeitig registrierten Integrationen und parallelen Clientzugriffen überprüft. Automatisierte Tests sollen sowohl die regulären Anwendungsfälle als auch relevante Fehler- und Nebenläufigkeitssituationen reproduzierbar abdecken. Ergänzend wird untersucht, welchen zusätzlichen Kommunikationsaufwand das Plugin verursacht und welche Anpassungen für die Anbindung eines bestehenden DeviceAgents notwendig sind.

Nicht Ziel der Arbeit ist es, die vorhandenen DeviceAgents oder deren gerätespezifische Kommunikation zu ersetzen. Auch die bestehende öffentliche Clientschnittstelle

soll nicht grundlegend verändert werden. Eine produktionsreife Migration sämtlicher DeviceAgents, eine persistierte oder verteilte Geräteverwaltung, umfassende Sicherheits- und Berechtigungskonzepte sowie eine vollständige produktive Gateway-Anbindung sind nicht Bestandteil des verpflichtenden Prototyps. Sie können jedoch als spätere Erweiterungen berücksichtigt werden.

1.4. Aufbau der Arbeit

Kapitel 2 erläutert die fachlichen und technischen Grundlagen. Dazu gehören die Rolle von zenLAB als Middleware und die Aufgaben der DeviceAgents sowie grundlegende Konzepte für verteilte Kommunikation, Registrierung und die Vermittlung von Aufträgen und Ereignissen. Ergänzend werden die relevanten Architekturprinzipien und Kommunikationstechnologien erklärt.

Kapitel 3 analysiert die bestehende Systemlandschaft und leitet daraus die funktionalen und nicht-funktionalen Anforderungen an das Device-Integration-Plugin ab. Zusätzlich werden der verbindliche Funktionsumfang des Prototyps und die Grenzen der Umsetzung festgelegt.

Kapitel 4 verbindet die Konzeption mit der prototypischen Implementierung und Einbindung in zenLAB. Entwurf und Umsetzung werden entlang der zentralen Themen des Plugins dargestellt: Architektur und Gerätemodell, Registrierung und Geräteverwaltung, Auftragsvermittlung, Schnittstellen, Anbindung bestehender DeviceAgents und Subscriptions. Abschließend werden die wesentlichen Herausforderungen der Umsetzung beschrieben.

Kapitel 5 evaluiert den Prototyp anhand der festgelegten Anforderungen und definierter Testszenarien. Betrachtet werden Funktionalität, Fehlerverhalten, gleichzeitige Zugriffe, Integrationsaufwand und zusätzlicher Kommunikationsaufwand. Kapitel 6 fasst die Ergebnisse zusammen, beantwortet die Forschungsfragen und zeigt mögliche Weiterentwicklungen auf.

Kapitel 2.

Grundlagen

Dieses Kapitel beschreibt die fachlichen und technischen Grundlagen, die zum Verständnis der weiteren Arbeit erforderlich sind. Zunächst werden zenLAB als Middleware und die Aufgaben der DeviceAgents erläutert. Anschließend folgen allgemeine Grundlagen verteilter Kommunikation, dynamischer Dienste und ereignisbasierter Datenübertragung. Den Abschluss bilden die relevanten Architekturprinzipien und Kommunikationstechnologien. Die Anforderungen an das Device-Integration-Plugin werden in Kapitel 3 hergeleitet; die Auswahl und Umsetzung der Lösung ist Gegenstand von Kapitel 4.

2.1. zenLAB

2.1.1. Middleware- und Plugin-Architektur

zenLAB ist ein von der infoteam Software AG entwickeltes Middleware-Framework für vernetzte digitale Labore [4]. Eine Middleware bildet eine vermittelnde Softwareschicht zwischen technisch unterschiedlichen Komponenten. Im betrachteten Anwendungsfall trennt sie die gerätenahe Kommunikation von den Anwendungen, die Gerätefunktionen verwenden. Dadurch muss eine Clientanwendung weder das herstellerspezifische Protokoll noch die konkrete Verbindung eines Laborgeräts kennen.

Die Kapselung technischer Unterschiede ist für Laborumgebungen besonders relevant. Geräte verschiedener Hersteller können ihre Funktionen über serielle Verbindungen, Netzwerkprotokolle, proprietäre Bibliotheken oder standardisierte Schnittstellen bereitstellen. Ohne eine gemeinsame Vermittlungsschicht müsste jede Anwendung die spezifischen Details der Gerätekommunikation implementieren. zenLAB abstrahiert diese Unterschiede und bietet eine einheitliche Schnittstelle für den Zugriff auf Gerätefunktionen.

Abbildung 2.1 ordnet eine solche Middleware zwischen der Geräteebene und übergeordneten Labor-, Planungs- und Managementsystemen ein. Die Middleware verbindet damit die gerätenahe Software mit Prozessleitsystemen, LIMS und ERP-Systemen. Gleichzeitig

bleibt sie an die jeweiligen Bedingungen eines Labors anpassbar und kann schrittweise um benötigte Funktionen erweitert werden [5].

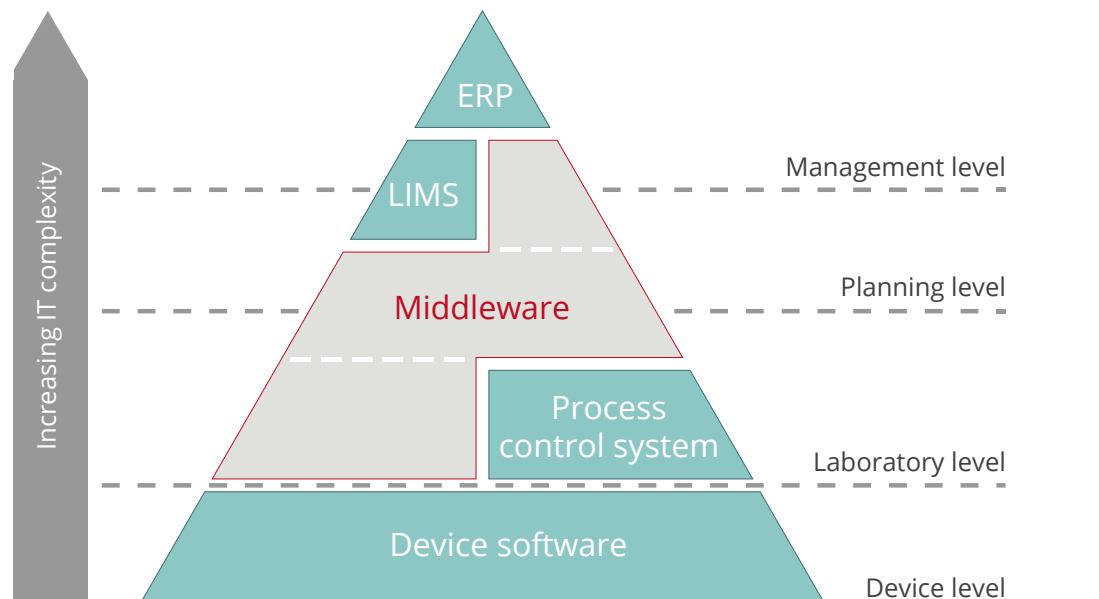


Fig. 1: Scalable customised middleware can be adapted to the particular conditions and requirements of each laboratory and flexibly expanded at any time. zenLAB supplies the necessary standard components.

Abbildung 2.1.: Einordnung einer skalierbaren Middleware zwischen Geräte-, Labor-, Planungs- und Managementebene (Quelle: infoteam Software AG [5, Abb. 1]).

Die Architektur von zenLAB ist modular aufgebaut. Abgrenzbare Funktionen werden in Plugins gekapselt und durch einen Host als Dienste bereitgestellt. Die Interaktion zwischen Komponenten erfolgt über definierte Schnittstellen und gemeinsam verwendete Datentypen. Verträge und Implementierungen können dadurch getrennt entwickelt werden. Ein aufrufendes Modul ist nicht von einer konkreten Implementierung abhängig, solange der vereinbarte Vertrag eingehalten wird.

Eine solche Plugin-Architektur erleichtert die Erweiterung des Gesamtsystems. Neue Funktionen können als zusätzliche Module eingebunden werden, ohne sämtliche bestehenden Komponenten anzupassen. Gleichzeitig erfordert die Modularisierung klar abgegrenzte Verantwortlichkeiten und kontrollierte Abhängigkeiten zwischen den einzelnen Plugins.

2.1.2. DeviceAgents

Ein DeviceAgent bildet die gerätenahe Integrationsschicht. Er kennt das Protokoll und die technischen Eigenschaften eines angebundenen Geräts und übersetzt zwischen dessen Schnittstelle und den von zenLAB erwarteten Funktionen. Zu seinen Aufgaben können

der Aufbau einer Geräteverbindung, das Lesen und Schreiben von Werten, das Ausführen von Befehlen sowie die Verarbeitung spontan eintreffender Geräteereignisse gehören.

Die gerätespezifische Logik verbleibt im jeweiligen DeviceAgent. Nur dort ist bekannt, wie ein konkretes Gerät adressiert wird, welche Datentypen es verwendet und wie Fehler des Geräteprotokolls zu interpretieren sind. Diese Abgrenzung ermöglicht es, Geräte unterschiedlicher Hersteller über eine gemeinsame übergeordnete Struktur anzusprechen, ohne ihre technischen Besonderheiten zu verlieren.

DeviceAgents können in eigenen Prozessen ausgeführt werden. Die Prozessgrenze erhöht die Isolation, weil ein Fehler in einer gerätespezifischen Implementierung nicht unmittelbar im zenLAB-Host auftreten muss. Gleichzeitig können über diese Grenze keine direkten Objektreferenzen verwendet werden. Aufrufe, Parameter, Ergebnisse und Ereignisse müssen daher durch übertragbare Nachrichten beschrieben werden.

2.2. Grundlagen verteilter Kommunikation

2.2.1. Auftrags- und Ergebnisprinzip

Bei einem lokalen Methodenaufruf befinden sich Aufrufer und ausführende Komponente üblicherweise im selben Prozess. Über eine Prozess- oder Netzwerkgrenze ist dies nicht möglich. Die gewünschte Operation muss in eine übertragbare Nachricht überführt und auf der Gegenseite wieder einer ausführbaren Funktion zugeordnet werden.

Ein auftragsbasiertes Modell beschreibt eine solche Nachricht als Auftrag beziehungsweise Job. Der Auftrag enthält eine Kennung, die gewünschte Operation, die benötigten Parameter und gegebenenfalls Informationen über das Ziel. Parameter müssen in einem Format dargestellt werden, das vom verwendeten Transport übertragen und auf der Gegenseite rekonstruiert werden kann.

Nach der Bearbeitung erzeugt die ausführende Komponente ein Ergebnis. Dieses verweist auf den ursprünglichen Auftrag und enthält entweder einen Rückgabewert oder eine definierte Fehlerdarstellung. Ausnahmen können nicht als lokale Laufzeitobjekte über eine Prozessgrenze weitergereicht werden. Relevante Fehlerinformationen müssen deshalb in einen transportierbaren Datentyp überführt werden.

Das Auftrags- und Ergebnisprinzip entkoppelt die fachliche Operation vom Transport. Der Aufrufer muss nicht wissen, in welchem Prozess oder durch welche konkrete Klasse ein Auftrag ausgeführt wird. Beide Seiten benötigen lediglich gemeinsame Verträge für Operationen, Parameter, Ergebnisse und Fehler.

2.2.2. Korrelation, Timeouts und Abbruch

Werden mehrere Aufträge parallel verarbeitet, treffen Ergebnisse nicht zwingend in derselben Reihenfolge ein, in der die Aufträge versendet wurden. Für eine korrekte Zuordnung benötigt daher jeder Auftrag eine eindeutige Kennung. Eine Korrelationskomponente merkt sich, welche wartende Operation zu welcher Auftragskennung gehört, und schließt beim Eintreffen eines Ergebnisses ausschließlich die passende Operation ab.

Ein Timeout begrenzt die Wartezeit auf ein Ergebnis. Bleibt eine Antwort aus, wird die wartende Operation mit einem definierten Fehler beendet und ihr Korrelationszustand entfernt. Abbruchsignale erlauben es außerdem, eine Operation zu beenden, wenn der Aufrufer seine Anfrage zurückzieht, eine Verbindung getrennt wird oder das Gesamtsystem herunterfährt.

Timeout und Abbruch beenden zunächst das lokale Warten. Die entfernte Verarbeitung kann zu diesem Zeitpunkt bereits begonnen haben. Ein Ergebnis kann deshalb noch eintreffen, nachdem der ursprüngliche Aufrufer den Vorgang bereits beendet hat. Verteilte Systeme benötigen für diesen Fall einen definierten Umgang mit verspäteten Nachrichten.

2.2.3. Adressierung und Routing

Routing bezeichnet die Auswahl des Empfängers, der eine Nachricht oder einen Auftrag bearbeiten soll. In einem System mit nur einer festen Gegenstelle kann das Ziel implizit sein. Sobald mehrere Dienste gleichzeitig aktiv sind, muss eine Nachricht ausreichende Zielinformationen enthalten oder anhand einer fachlichen Identität auf einen Empfänger aufgelöst werden können.

Beim Unicast-Routing wird ein Auftrag an genau ein Ziel übertragen. Ein Broadcast richtet sich dagegen an mehrere Empfänger. Bei einem Broadcast können mehrere Ergebnisse entstehen, die zu unterschiedlichen Zeitpunkten eintreffen oder teilweise ausbleiben. Die Aggregation solcher Teilergebnisse ist deshalb aufwendiger als eine einfache Anfrage-Antwort-Zuordnung.

2.2.4. Asynchrone und bidirektionale Streams

Neben einzelnen Anfrage-Antwort-Aufrufen können verteilte Systeme länger bestehende Kommunikationswege benötigen. Eine bidirektionale Verbindung ermöglicht, dass beide Seiten unabhängig voneinander Nachrichten übertragen, ohne für jedes Element eine neue Verbindung aufzubauen.

Asynchrone Streams modellieren eine Folge von Elementen, deren Umfang und Ankunftszeit beim Start noch nicht vollständig bekannt sind. In C# können solche Folgen mit `IAsyncEnumerable<T>` beschrieben und mit `await foreach` verarbeitet werden. Dabei wird jedes Element verarbeitet, sobald es verfügbar ist, ohne während der gesamten Wartezeit einen Thread zu blockieren [6].

Eine länger bestehende Verbindung muss kontrolliert beendet werden können. Streamende, Transportfehler und Abbruchsignale sind deshalb Bestandteile ihres Lebenszyklus. Wenn ein Dienst seine Funktionen über mehrere dauerhaft geöffnete Streams bereitstellt, kann seine Verfügbarkeit an den Zustand dieser Streams gekoppelt werden. Dann muss festgelegt sein, welche Streams erforderlich sind und welche Zustandsübergänge den Dienst als nicht mehr verfügbar kennzeichnen.

2.3. Dynamische Dienste und Ereignisse

2.3.1. Registrierung

Dynamisch verfügbare Dienste müssen einem Gesamtsystem mitteilen, dass sie erreichbar sind und welche Funktionen oder Ressourcen sie bereitstellen. Dieser Vorgang wird als Registrierung bezeichnet. Eine zentrale Verwaltung kann anschließend Auskunft über die aktuell bekannten Dienste geben und wird damit zu einer Grundlage für Service Discovery und Routing. Auch andere Ansätze zur Laborautomation verwenden eine zentrale Verwaltung verfügbarer Geräte und Dienste, um deren Funktionen auffindbar zu machen [2].

Eine logische Dienstidentität ist von einer konkreten Verbindung zu unterscheiden. Die logische Identität kann über Neustarts hinweg bestehen bleiben, während einzelne Verbindungen zeitlich begrenzt sind. Diese Unterscheidung ist allgemein relevant, wenn verspätete Nachrichten einer vorherigen Verbindung nicht mit dem Zustand einer neuen Verbindung vermischt werden dürfen.

2.3.2. Ereignisse und Abonnements

Nicht jede Information wird durch einen Aufrufer aktiv abgefragt. Zustände können sich unabhängig von einer Anfrage verändern. Polling würde erfordern, dieselben Werte in regelmäßigen Abständen erneut zu lesen. Dies erzeugt wiederholte Aufrufe und erkennt Änderungen nur mit einer vom Abfrageintervall abhängigen Verzögerung.

Ein Abonnement beschreibt das fortlaufende Interesse eines Empfängers an einer bestimmten Ereignisquelle. Zwischen Einrichtung und Beendigung können beliebig viele

Ereignisse eintreffen. Der Lebenszyklus eines Abonnements unterscheidet sich daher von einer einmaligen Anfrage-Antwort-Kommunikation. Nach seinem Ende müssen die dafür belegten Kommunikations- und Verwaltungsressourcen wieder freigegeben werden.

Produziert eine Ereignisquelle dauerhaft mehr Elemente, als ein Empfänger verarbeiten kann, entsteht Backpressure. Ein unbegrenzter Puffer würde in diesem Fall kontinuierlich wachsen. Begrenzte Warteschlangen machen den maximalen Ressourcenbedarf kontrollierbar. Welche Strategie bei einem gefüllten Puffer geeignet ist, hängt davon ab, ob jedes Ereignis erhalten bleiben muss oder nur der jeweils aktuelle Zustand relevant ist.

2.4. Architektur- und Technologiegrundlagen

2.4.1. Stabile Verträge und Modularisierung

Eine Plugin-Architektur trennt einen stabilen Kern von ergänzbaren Funktionsbereichen. Die Interaktion erfolgt über definierte Erweiterungspunkte und Verträge. Solche Architekturen unterstützen eine unabhängige Entwicklung und Bereitstellung einzelner Komponenten, erfordern jedoch eine kontrollierte Verwaltung von Schnittstellen und Abhängigkeiten [7].

Verträge beschreiben, welche Operationen eine Komponente anbietet und welche Daten dabei ausgetauscht werden. Sie sollten nur Informationen enthalten, die für die Kommunikation tatsächlich erforderlich sind. Interne Implementierungsdetails gehören nicht in öffentliche Contracts, weil Änderungen daran sonst unnötig viele abhängige Komponenten betreffen würden.

2.4.2. gRPC und SignalR

gRPC eignet sich für klar definierte Dienstgrenzen zwischen verteilten Komponenten. Es unterstützt typisierte Anfrage-Antwort-Aufrufe sowie Client-, Server- und bidirektionales Streaming. Beim code-first-Ansatz werden Dienst- und Nachrichtenverträge durch .NET-Schnittstellen und Datentypen beschrieben. Client und Server können dadurch gemeinsame Verträge verwenden, ohne eine manuell gepflegte `.proto`-Datei als primäre Quelle zu benötigen [8].

SignalR stellt eine Hub-basierte Echtzeitkommunikation für verbundene Clients bereit. Der Server kann Nachrichten gezielt an einzelne Verbindungen oder Gruppen senden. Darüber hinaus unterstützt SignalR Streaming vom Client zum Server und vom Server

zum Client. Elemente können übertragen werden, sobald sie verfügbar sind, anstatt zunächst eine vollständige Ergebnismenge zu sammeln [9, 10].

gRPC und SignalR können damit unterschiedliche Kommunikationsgrenzen bedienen. Unabhängig von der Transporttechnologie benötigt eine Anwendung eigene Regeln für Identität, Lebenszyklus, Korrelation und Fehlerbehandlung.

2.4.3. Dependency Injection und Testbarkeit

Dependency Injection bezeichnet die Bereitstellung von Abhängigkeiten von außen, anstatt sie innerhalb einer Klasse fest zu erzeugen. Dadurch hängt eine Komponente von einem Vertrag und nicht zwingend von einer konkreten Implementierung ab. In .NET sind Dependency Injection und ein zugehöriger Servicecontainer Bestandteil des Frameworks [11].

Werden Abhängigkeiten über Konstruktoren bereitgestellt, bleiben sie sichtbar und können in automatisierten Tests durch kontrollierte Testimplementierungen ersetzt werden. Dies unterstützt insbesondere die Prüfung zeitabhängiger und nebenläufiger Abläufe. Dependency Injection allein garantiert jedoch noch keine klare Struktur. Eine große Zahl von Abhängigkeiten kann darauf hinweisen, dass eine Klasse mehrere Verantwortlichkeiten übernimmt.

2.5. Zusammenfassung

Das bestehende zenLAB-System stellt mit den DeviceAgents die fachliche und technische Ausgangsbasis der Arbeit bereit. Für die weitere Untersuchung sind außerdem die allgemeinen Prinzipien verteilter Kommunikation, dynamischer Registrierung und ereignisbasierter Datenübertragung relevant.

Die dargestellten Architektur- und Technologiegrundlagen erklären die später verwendeten Begriffe und Mechanismen, ohne bereits Anforderungen oder konkrete Lösungsentscheidungen für das Device-Integration-Plugin festzulegen. Auf dieser Grundlage analysiert das folgende Kapitel die Ausgangssituation und leitet die Anforderungen an das Zielsystem her.

Kapitel 3.

Anforderungsanalyse

3.1. Ausgangssituation

3.1.1. Analyse der bestehenden Kommunikationsarchitektur

DeviceAgents stellen die Funktionen der angebundenen Geräte bereit und können in eigenen Prozessen ausgeführt werden. Ein gestarteter Prozess ist jedoch nicht zwangsläufig bei einer zentralen Integrationskomponente registriert und erreichbar. Umgekehrt kann eine Kommunikationsverbindung unterbrochen sein, obwohl der Prozess weiterhin ausgeführt wird.

Das zu entwickelnde Plugin muss feststellen können, welche Integrationen sich aktuell angemeldet haben, welche Geräte sie bereitstellen und ob ihre Kommunikationsverbindung noch gültig ist. Die gerätespezifische Kommunikation verbleibt dabei in den bestehenden DeviceAgents.

Clientanwendungen greifen über eine bestehende Clientbibliothek auf die WebAPI zu. Auftragsbezogene Operationen werden über HTTP und ereignisbasierte Informationen über SignalR übertragen. Diese bereits verwendete Systemgrenze bildet den Ausgangspunkt für die Einordnung des Device-Integration-Plugins.

3.1.2. Grenzen der bisherigen Serviceanbindung

Mit mehreren gleichzeitig aktiven DeviceAgents kann nicht mehr davon ausgegangen werden, dass jeder Auftrag automatisch an denselben Empfänger übertragen wird. Es muss bekannt sein, welche Integrationen und Geräte aktuell erreichbar sind und über welche Verbindung ein bestimmtes Gerät angesprochen werden kann.

Prozessstart und Prozessüberwachung allein reichen hierfür nicht aus. Die für eine Integration erforderlichen Kommunikationsstreams können regulär enden, abgebrochen werden oder aufgrund eines Fehlers ausfallen, obwohl der zugehörige Prozess weiterhin ausgeführt wird. Solche Zustandsänderungen und eine erneute Anbindung müssen

deshalb unabhängig vom Betriebssystemprozess behandelt werden. Gleichzeitig darf ein beendeter Stream einer vorherigen Verbindung den Zustand neu geöffneter Streams nicht beeinflussen.

3.2. Zielsystem und Rahmenbedingungen

3.2.1. Beteiligte Akteure und Komponenten

Eine Integration bezeichnet eine eigenständig kommunizierende Softwarekomponente, die dem Plugin ein oder mehrere Geräte zur Verfügung stellt. Ein DeviceAgent kann eine solche Integration bilden oder an eine entsprechende Laufzeitkomponente angebunden werden. Ein Gerät bezeichnet dagegen die fachliche Einheit, deren Funktionen von Clients verwendet werden.

Eine Integration besitzt eine logische Identität, über die sie systemweit unterschieden werden kann. Ihre aktuelle Verfügbarkeit ergibt sich dagegen aus den für sie geöffneten Kommunikationsstreams. Nach einem Neustart oder einer erneuten Verbindung kann dieselbe logische Integration diese Streams neu aufbauen.

3.2.2. Bestehende Client- und WebAPI-Grenze

Clientbibliothek und WebAPI sollen als bestehende Systemgrenze erhalten bleiben. Die WebAPI bleibt für webbezogene Aufgaben zuständig, während das Device-Integration-Plugin hinter dieser Grenze die Verwaltung und Vermittlung der Integrationen und Geräte übernimmt.

3.2.3. Minimaler Funktionsumfang

3.3. Funktionale Anforderungen

3.3.1. Verfügbarkeit und eindeutige Identifikation

Dynamisch verfügbare Integrationen müssen sich gegenüber dem System eindeutig identifizieren und die für ihre Funktionen erforderlichen Methodenstreams öffnen. Eine Integration darf erst dann als verfügbar geführt werden, wenn sämtliche verpflichtenden Streams aktiv sind. Die logische Identität muss dabei unabhängig von einer einzelnen Streaminstanz erhalten bleiben.

Auch Geräte benötigen eine eindeutige Identität. Ein Client soll ein Gerät unabhängig davon adressieren können, über welche technische Verbindung seine Integration gerade kommuniziert. Zustandsänderungen vorheriger Streaminstanzen dürfen nicht als Zustandsänderungen neu geöffneter Streams derselben Integration behandelt werden.

3.3.2. Verwaltung von Integrationen und Devices

Das Plugin muss eine aktuelle Sicht auf die verfügbaren Integrationen und deren Geräte bereitstellen. Es muss ermitteln können, ob die erforderlichen Methodenstreams einer Integration aktiv sind und welche Geräte über sie erreichbar sind.

Gerätelisten sind nicht notwendigerweise statisch. Eine Integration kann während ihrer Laufzeit weitere Geräte erkennen oder bisher verfügbare Geräte verlieren. Änderungen der Geräteliste müssen daher konsistent übernommen werden. Streamzustände, Geräteaktualisierungen, Clientzugriffe und Aufräumvorgänge können gleichzeitig auftreten; ein Client darf kein Gerät erhalten, dessen Integration bereits als nicht verfügbar entfernt wurde.

3.3.3. Methodenstreams, Verbindungsabbruch und erneute Anbindung

Die Verfügbarkeit einer Integration muss an den Lebenszyklus ihrer erforderlichen Methodenstreams gekoppelt werden. Endet einer dieser Streams regulär, wird er abgebrochen oder schlägt er mit einem Fehler fehl, darf die Integration nicht länger als erreichbares Routingziel geführt werden.

Beim Verlust der Verfügbarkeit müssen die zugehörigen Gerätezuordnungen, offenen Aufträge, Kommunikationskanäle und Subscriptions bereinigt werden. Öffnet dieselbe logische Integration ihre erforderlichen Methodenstreams erneut, darf ein nachträglicher Zustandswechsel einer vorherigen Streaminstanz den neuen Zustand nicht verändern.

3.3.4. Routing von Client-Aufträgen

Jeder gerätebezogene Auftrag muss an die aktuell zuständige Integration vermittelt werden können. Ist das Gerät unbekannt, die Integration nicht aktiv oder keine gültige Route mehr vorhanden, muss der Zugriff mit einem nachvollziehbaren Fehler beendet werden.

Mehrere parallele Aufträge und ihre Ergebnisse müssen eindeutig zugeordnet werden können. Ergebnisse mit unbekannter oder bereits abgeschlossener Auftragskennung dürfen nicht einem anderen Auftrag zugewiesen werden.

3.3.5. SmartProxy und gRPC-Servicevertrag

3.3.6. Bereitstellung von Gerätefunktionen

Bestehende DeviceAgents sollen weiterverwendet werden. Clients müssen verfügbare Gerätefunktionen ermitteln, Werte lesen und setzen, Methoden aufrufen sowie Abonnements anlegen und beenden können. Die fachliche Bedeutung der Gerätefunktionen und die eigentliche Hardwarekommunikation verbleiben im DeviceAgent. Welche Darstellung und Schnittstelle diese Zugriffe innerhalb des Plugins unterstützen, wird im Lösungskonzept begründet.

3.3.7. Abonnieren von Wertänderungen

Clients müssen Geräteänderungen abonnieren und wieder abbestellen können. Zwischen Einrichtung und Beendigung eines Abonnements können mehrere Ereignisse eintreffen, die dem jeweils interessierten Client zugeordnet werden müssen.

Mehrere Clients können dieselbe Ereignisquelle abonnieren. Endet eine Clientverbindung oder wird eine Integration aufgrund eines beendeten Methodenstreams entfernt, müssen die zugehörigen Abonnement- und Kommunikationszustände freigegeben werden.

3.4. Nicht-funktionale Anforderungen

3.4.1. Wiederverwendbarkeit

Gemeinsam verwendete Auftrags-, Ergebnis- und Kommunikationsverträge sollen nicht von der Implementierung eines einzelnen DeviceAgents abhängig sein. Die technische Vermittlung darf keine gerätespezifische Fachlogik enthalten.

3.4.2. Erweiterbarkeit

3.4.3. Zuverlässigkeit

Timeouts und Abbrüche müssen wartende Operationen kontrolliert beenden. Verspätete Ergebnisse dürfen bereits abgeschlossene Abläufe nicht nachträglich verändern. Aufräumvorgänge müssen auch bei mehreren nahezu gleichzeitig auftretenden Ursachen zu einem konsistenten Endzustand führen.

3.4.4. Nebenläufigkeit

Gemeinsam verwendete Stream-, Verfügbarkeits-, Geräte- und Abonnementzustände müssen bei parallelen Zugriffen konsistent bleiben. Eine langsame Clientverbindung darf die Verarbeitung für andere Empfänger nicht unbegrenzt blockieren. Für Ereignisse derselben Quelle muss eine definierte Reihenfolge gewährleistet werden können.

3.4.5. Performance

Der Ressourcenbedarf für länger bestehende Streams und Ereignispuffer muss begrenzt werden können. Wenn eine Ereignisquelle dauerhaft mehr Elemente erzeugt, als ein Empfänger verarbeitet, darf dies nicht zu einem unbegrenzten Anwachsen des Speicherverbrauchs führen.

3.5. Abgrenzung

3.6. Zusammenfassung der Anforderungen

Kapitel 4.

Konzeption und Implementierung

Dieses Kapitel verbindet die Entwurfsentscheidungen für das Device-Integration-Plugin mit ihrer prototypischen Umsetzung. Die Darstellung folgt den Verantwortungsbereichen des Plugins, sodass die jeweilige Lösung und ihre Implementierung gemeinsam erläutert werden können.

4.1. Ziel und Gesamtarchitektur

Für das Device-Integration-Plugin sind mehrere fachliche Bereiche zu unterscheiden: die Kommunikation mit Clients, die Kommunikation mit Integrationen, die Verwaltung ihrer Verfügbarkeit und Geräte sowie die Verteilung von Ereignissen. Ihre Verantwortlichkeiten sollen nicht in einer einzigen allzuständigen Komponente vermischt werden.

4.2. Technologische Rahmenbedingungen und Projektstruktur

4.2.1. Technologische Rahmenbedingungen

4.2.2. Projekt- und Paketstruktur

Die Projekt- und Paketstruktur bildet die fachlichen und technischen Grenzen der Lösung ab. Allgemeine Contracts bleiben unabhängig von den konkreten Implementierungen der Geräteanbindung. Die Komponenten erhalten ihre Abhängigkeiten über definierte Verträge und Dependency Injection. Dadurch können sie automatisiert mit kontrollierten Testimplementierungen geprüft werden.

4.3. Auswahl und Nutzung des Gerätemodells

Die Anforderungen beschreiben zunächst die benötigten Gerätezugriffe: Funktionen ermitteln, Werte lesen und setzen, Methoden aufrufen sowie Änderungen abonnieren. Für die Umsetzung ist zu klären, wie diese Funktionen unabhängig vom jeweiligen Geräteprotokoll beschrieben und adressiert werden können.

Mit dem `TreeNode`-Modell steht in der bestehenden zenLAB-Umgebung bereits ein entsprechender Ansatz zur Verfügung. Es ordnet Gerätefunktionen hierarchisch in Knoten an, die über Pfade erreichbar sind. Knoten können Werte, Methoden oder strukturelle Teile des Geräts repräsentieren. Die zugehörige Schnittstelle unterstützt außerdem das Ermitteln von Knoteninformationen und das Abonnieren von Änderungen.

Die vorhandenen Operationen passen damit zu den benötigten Gerätezugriffen. Für die prototypische Anbindung bietet es sich an, diesen Ansatz über einen Adapter weiterzuverwenden, statt ein zusätzliches Gerätemodell einzuführen. Gegenstand der Arbeit ist dabei die Auswahl und Einbindung des vorhandenen Modells in das neue Plugin, nicht die Entwicklung des `TreeNode`-Modells selbst. Verfügbarkeitsüberwachung, Geräteverwaltung und Kommunikationslebenszyklus bleiben davon getrennte Aufgaben des Plugins.

4.4. Verfügbarkeits- und Lebenszyklusverwaltung

4.4.1. Service Directory und Gerätecatalog

Ein zentrales Service Directory verwaltet die aktuell verfügbaren Integrationen anhand ihrer `ServiceIdentification`. Diese logische Identität dient als Routingziel und bleibt unabhängig von einzelnen Kommunikationsstreams. Eine Integration wird in das Service Directory aufgenommen, sobald alle für sie erforderlichen Methodenstreams aktiv sind. Damit bildet nicht eine separate zeitlich begrenzte Registrierung, sondern der aktuelle Streamzustand die Grundlage der Verfügbarkeit.

Der `IntegrationDeviceCatalog` verwaltet davon getrennt die über jede Integration verfügbaren Geräte. Wird eine Integration verfügbar, fordert eine Synchronisationskomponente über den Auftragsmechanismus ihre Geräteliste an. Das Ergebnis wird als vollständiger Snapshot übernommen. Bei der Geräteidentifikation wird die Identität der Integration mit der Geräte-ID kombiniert, sodass gleiche lokale Geräte-IDs verschiedener Integrationen unterscheidbar bleiben.

Wird eine Integration aus dem Service Directory entfernt, bricht der Synchronizer eine gegebenenfalls noch laufende Abfrage ab und entfernt alle ihr zugeordneten Geräte aus

dem Katalog. Zugriffe auf Service Directory und Gerätekatalog werden jeweils synchronisiert, damit parallele Streamzustände, Geräteabfragen und Aufräumvorgänge konsistent verarbeitet werden.

4.4.2. Methodenstream-Tracking

Für die Kommunikation öffnet eine Integration länger bestehende Methodenstreams, über die sie Aufträge des Plugins entgegennimmt. Der Prototyp definiert den Request-Stream der Operation `GetDevices` als erforderlichen Stream. Weitere Geräteoperationen können durch zusätzliche `StreamDefinition`-Einträge in dieselbe Überwachung aufgenommen werden.

Ein `TrackedStream<T>` dekoriert den zugrunde liegenden asynchronen Stream und veröffentlicht dessen Lebenszykluszustand. Mit Beginn der Enumeration wechselt er in den Zustand `Active`. Ein reguläres Streamende führt zu `Completed`; ein Abbruch zu `Canceled` und eine Ausnahme zu `Faulted`. Jeder getrackte Stream darf nur einmal enumeriert werden, damit sein Zustand eindeutig einer Verbindung zugeordnet bleibt.

Der `ServiceStreamTracker` fasst die getrackten Streams einer Integration zusammen. Erst wenn alle als erforderlich definierten Streams aktiv sind, meldet er die Integration als verfügbar. Jeder terminale Zustand eines erforderlichen Streams entzieht diese Verfügbarkeit wieder. Der `StreamTrackerCoordinator` überführt die Zustandsänderungen in die Registrierung beziehungsweise Entfernung des Dienstes in der Routing-Registry.

Die Einbindung in die Auftragsvermittlung erfolgt durch den Decorator `TrackedJobTransmitter`. Beim Öffnen eines Request-Streams übergibt er den Stream zusammen mit der Dienstidentität und der zugehörigen Streamdefinition an den Coordinator. Die fachliche Auftragsverarbeitung bleibt dadurch vom Tracking des Kommunikationslebenszyklus getrennt.

4.4.3. Streamabbruch, erneute Anbindung und Bereinigung

Sobald ein erforderlicher Stream endet, abgebrochen wird oder fehlschlägt, entfernt der Coordinator die betroffene Integration aus der Routing-Registry und verwirft ihren `StreamTracker`. Die Entfernung löst über das Service Directory die Bereinigung der zugehörigen Geräte aus. Andere Integrationen besitzen eigene Tracker und bleiben von diesem Vorgang unberührt.

Öffnet dieselbe logische Integration ihre erforderlichen Streams später erneut, wird für sie ein neuer Tracker erzeugt. Bei der Verarbeitung eines Zustandswechsels prüft der Coordinator neben der Dienstidentität auch, ob die Meldung vom aktuell eingetragenen

Tracker stammt. Dadurch kann ein verspäteter Callback einer vorherigen Streaminstanz keinen neu aufgebauten Zustand entfernen.

4.5. Auftragsvermittlung und Multi-Service-Routing

4.5.1. Routing-Grundlage

Allgemeine Auftrags-, Ergebnis-, Routing- und Stream-Contracts werden von den fachlichen Geräteoperationen getrennt. Der Routing-Kern übernimmt Übertragung, Zielauflösung, Korrelation und Timeoutbehandlung. Das Device-Integration-Plugin ergänzt darauf aufbauend die Verwaltung der Integrationen und Geräte, deren Lebenszyklus sowie die Übersetzung zu den bestehenden Gerätefunktionen.

4.5.2. Serviceidentifikation und Routenauflösung

Der Gerätecatalog liefert die Zuordnung zwischen einem Gerät und der aktuell zuständigen Integration. Die Routingkomponente löst die aktive Route zum Sendezeitpunkt auf und übergibt den Auftrag an den zugehörigen Kommunikationskanal. Ist keine gültige Route vorhanden, wird der Auftrag mit einem definierten Fehler beendet.

Beim Unicast-Routing wird ein Auftrag an genau eine adressierte Integration weitergeleitet. Ein Broadcast erzeugt dagegen Teilaufträge für mehrere aktive Integrationen. Die zugehörigen Teilergebnisse werden unter Berücksichtigung einzelner Fehler und eines Gesamttimeouts zu einem Ergebnis zusammengeführt.

4.5.3. Job-Korrelation

Für einen wartenden Auftrag wird ein Korrelationszustand angelegt. Ein eingehendes Ergebnis wird nur dann angenommen, wenn Auftragskennung und logische Integration mit dem erwarteten Auftrag übereinstimmen. Ergebnisse mit unbekannter oder bereits abgeschlossener Kennung werden verworfen und können für Diagnosezwecke protokolliert werden.

4.5.4. Fehler- und Timeoutbehandlung

Ein konfigurierbares Timeout begrenzt die Wartezeit auf ein Ergebnis und entfernt anschließend den zugehörigen Korrelationszustand. Trifft ein Ergebnis verspätet ein, kann es keinem gültigen Auftrag mehr zugeordnet werden und verändert den bereits abgeschlossenen Ablauf nicht.

4.5.5. Implementierung der Routing-Grundlage und des Multi-Service-Routings

4.6. Contracts und Dienstschnittstellen

4.6.1. Job-, Ergebnis- und Stream-Contracts

Transportbezogene und fachliche Datentypen werden voneinander getrennt. Allgemeine Contracts beschreiben Aufträge, Ergebnisse, Routinginformationen und Streams. Die Contracts für Gerätezugriffe enthalten dagegen Gerätepfade, Werte, Methodenparameter und Ereignisse. Interne Implementierungsdetails werden nicht Bestandteil der öffentlich verwendeten Verträge.

4.6.2. Serviceverträge und SmartProxys

Der SmartProxy stellt der aufrufenden Komponente ein lokales Objekt mit demselben fachlichen Vertrag wie der entfernte Dienst zur Verfügung. Er kapselt Verbindungsaufbau, Serialisierung und Remote-Aufruf gegenüber dem Aufrufer.

4.6.3. Code-first gRPC und Umsetzung der Dienstschnittstellen

Für die interne Kommunikation wird code-first gRPC eingesetzt. Dadurch können die .NET-Komponenten gemeinsame Dienst- und Nachrichtenverträge verwenden. Neben einzelnen Anfrage-Antwort-Aufrufen werden die für länger bestehende Kommunikationswege erforderlichen Streamingformen unterstützt.

4.7. Integration Client und Anbindung bestehender DeviceAgents

4.7.1. Adapter für das gewählte Gerätemodell

Das Adapter-Prinzip ermöglicht die Verwendung einer bestehenden Komponente über eine im neuen Kontext benötigte Schnittstelle [12]. Für das gewählte Gerätemodell nimmt der Integrationsadapter transportierte Aufträge entgegen und bildet sie auf die vorhandene Schnittstelle `ITreeNodeBasedDeviceAgent` ab. Nach der Ausführung übersetzt er Ergebnisse, Fehler und Ereignisse zurück in die gemeinsamen Kommunikationsverträge. Der Adapter enthält keine gerätespezifische Fachlogik; diese verbleibt im angebundenen DeviceAgent.

4.7.2. Implementierung des Integration Clients und des Adapters

4.8. Subscriptions und Ereignisweitergabe

Für eine von mehreren Clients abonnierte Ereignisquelle wird ein gemeinsames vorgelagertes Abonnement eingerichtet. Weitere Clients werden als zusätzliche Empfänger derselben Quelle registriert. Eingehende Ereignisse werden anschließend an alle aktuell zugeordneten Clients verteilt.

Eine Referenzzählung bestimmt, wann das vorgelagerte Abonnement nicht mehr benötigt wird. Beendet ein Client sein Interesse, wird zunächst nur seine Zuordnung entfernt. Erst wenn kein Client mehr verbleibt, wird auch das Abonnement gegenüber der Integration aufgelöst.

Getrennte und begrenzte Warteschlangen isolieren langsame Clientverbindungen voneinander und begrenzen den maximalen Speicherverbrauch. Die jeweilige Pufferstrategie richtet sich danach, ob jedes Ereignis vollständig erhalten bleiben muss oder nur der aktuelle Zustand relevant ist. Für Ereignisse derselben Subscription bleibt die Reihenfolge erhalten.

4.8.1. Prototypische Umsetzung der Subscriptions

4.9. Einbindung in zenLAB und Referenzintegration

4.9.1. Einbindung in zenLAB

4.9.2. Anbindung eines Referenz-DeviceAgents

4.10. Herausforderungen während der Umsetzung

4.11. Begründung zentraler Entwurfsentscheidungen

Kapitel 5.

Evaluation

5.1. Ziel der Evaluation

5.2. Versuchsaufbau und Evaluationskriterien

5.3. Überprüfung der funktionalen Anforderungen

5.3.1. Verfügbarkeit und eindeutige Adressierung

5.3.2. Routing mit mehreren Integrationen und Clients

5.3.3. Aktivierung und Abbruch erforderlicher Methodenstreams

5.3.4. Gerätezugriffe und Subscriptions

5.3.5. Kompatibilität der bestehenden Client- und WebAPI-Grenze

5.4. Bewertung der Wiederverwendbarkeit und Erweiterbarkeit

5.5. Messung des Routing-Overheads

5.6. Bewertung anhand des Referenz-DeviceAgents

5.7. Diskussion der Ergebnisse

5.8. Grenzen der Evaluation

Kapitel 6.

Fazit und Ausblick

6.1. Zusammenfassung der Ergebnisse

6.2. Beantwortung der Forschungsfragen

6.3. Bewertung der Zielerreichung

6.4. Limitationen der Arbeit

6.5. Ausblick auf mögliche Erweiterungen

Anhang A.

Ergänzende zenLAB-Architektur

Abbildung [A.1](#) zeigt ergänzend die Architektur aus dem Whitepaper. Dargestellt ist das Zusammenspiel von zenLAB Essentials, Plugin-Host, Basiskomponenten, spezifischen Modulen, externen Systemen und angebundenen Geräten [\[5\]](#).

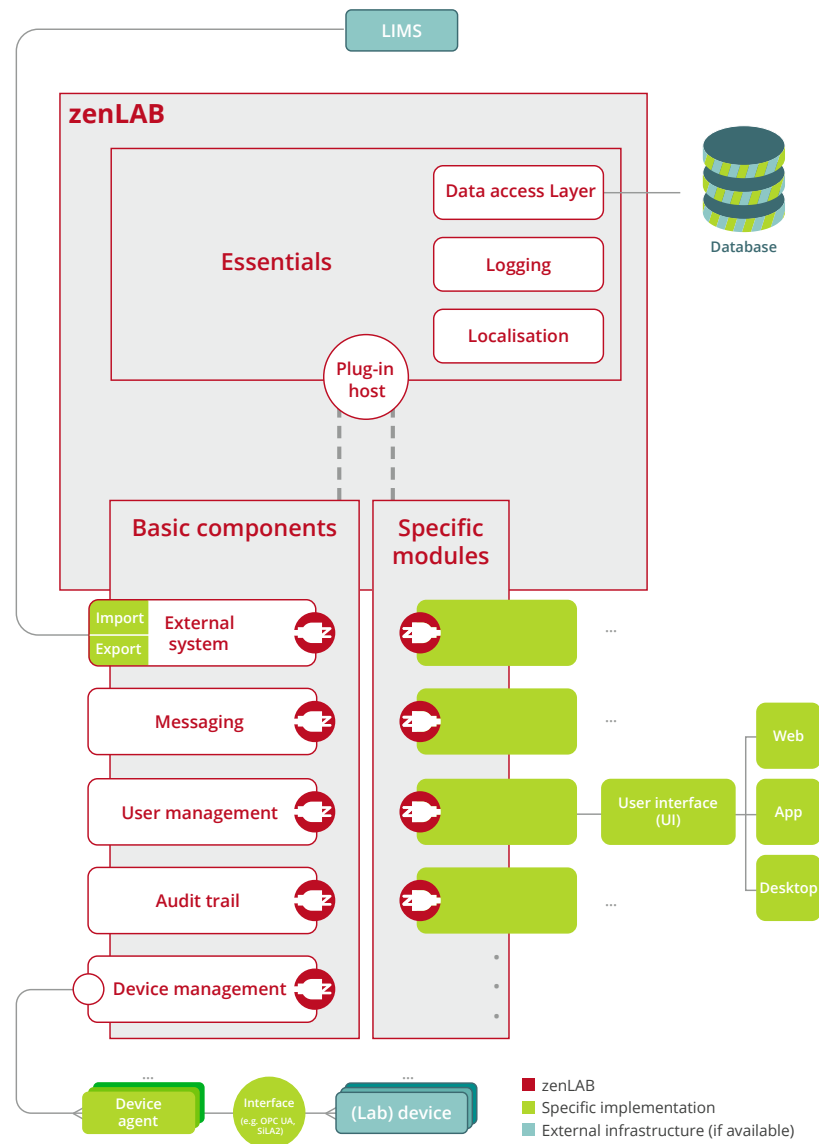


Abbildung A.1.: Gesamtarchitektur von zenLAB mit Essentials, Plugin-Host, Basiskomponenten und spezifischen Modulen (Quelle: infoteam Software AG [5, Abb. 2]).

Abbildungsverzeichnis

2.1. Einordnung einer skalierbaren Middleware	8
A.1. Gesamtarchitektur von zenLAB	30

Tabellenverzeichnis

List of Listings

Literaturverzeichnis

- [1] K. Janning, F. Kaußmann, J. Krieger, J. Mathews, F. Nienhaus, D. Reibert, C. Stehmar, and C. Skazik-Voogt, “Labor 4.0 – next generation production in life sciences,” Fraunhofer-Institut für Produktionstechnologie IPT, Trendreport, 2025. [Online]. Available: <https://www.ipt.fraunhofer.de/content/dam/ipt/de/documents/whitepaper/fraunhofer-ipt-publikation-trendreport-labor-40-next-generation-production-in-life-science1.pdf>
- [2] L. Bromig, D. Leiter, A.-V. Mardale, N. von den Eichen, E. Bieringer, and D. Weuster-Botz, “The SiLA 2 manager for rapid device integration and workflow automation,” *SoftwareX*, vol. 17, p. 100991, 2022.
- [3] infoteam Software AG, “Unternehmen,” [Online]. Verfügbar: <https://infoteam.de/unternehmen>, zugriff am 31. Juli 2026.
- [4] —, “Life Science,” [Online]. Verfügbar: <https://infoteam.de/unsere-maerkte/life-science>, zugriff am 31. Juli 2026.
- [5] —, “zenLAB – Middleware Framework for Networked Laboratories,” infoteam Software AG, Whitepaper, 2020. [Online]. Available: https://infoteam.de/fileadmin/user_upload/downloads/WP/20200904FD_b_zenLab_EN_ANSICHT_.pdf
- [6] Microsoft, “Generate and consume async streams,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/generate-and-consume-asynchronous-stream>, zugriff am 24. Juli 2026.
- [7] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester, UK: Wiley, 1996, vol. 1.
- [8] Microsoft, “Code-first grpc services and clients with .net,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/aspnet/core/grpc/code-first>, zugriff am 24. Juli 2026.
- [9] —, “Use hubs in asp.net core signalr,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/aspnet/core/signalr/hubs>, zugriff am 24. Juli 2026.

- [10] —, “Use streaming in asp.net core signalr,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/aspnet/core/signalr/streaming>, zugriff am 24. Juli 2026.
- [11] —, “Dependency injection in .net,” [Online]. Verfügbar: <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/overview>, zugriff am 24. Juli 2026.
- [12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

Glossar

library A suite of reusable code inside of a programming language for software development. i

shell Terminal of a Linux/Unix system for entering commands. i